

3.1 General Method

- In divide and conquer method, a given problem is,
 - i) Divided into smaller sub problems.
 - ii) These sub problems are solved independently.
 - iii) Combining all the solutions of sub problems into a solution of the whole.
- If the sub problems are large enough then divide and conquer is reapplied.
- The generated sub problems are usually of same type as the original problem. Hence recursive algorithms are used in divided and conquer strategy.
- A **control abstraction for divide and conquer** is as given below - using control abstraction a flow of control of a procedure is given.

Algorithm DC(P)

```
{  
    if P is too small then  
        return solution of P.  
    else  
    {  
        Divide (P) and obtain  $P_1, P_2, \dots, P_n$   
        where  $n \geq 1$   
  
        Apply DC to each subproblem  
        return combine (DC( $P_1$ ), DC( $P_2$ )... (DC( $P_n$ )));  
    }  
}
```

- The computing time of above procedure of divide and conquer is given by the recurrence relation.

$$T(n) = \begin{cases} g(n) & \text{if } n \text{ is small} \\ T(n_1) + T(n_2) + \dots + T(n_r) + F(n) & \text{when } n \text{ is sufficiently large} \end{cases}$$

Where $T(n)$ is the time for divide and conquer of size n . The $g(n)$ is the computing time required to solve small inputs. The $F(n)$ is the time required in dividing problem P and combining the solutions to sub problems. Let us now discuss some applications of this method.

The divide and conquer technique is as shown by Fig. 3.1.1.

The generated sub problems are usually of same type as the original problem. Hence sometimes recursive algorithms are used in divide and conquer strategy.

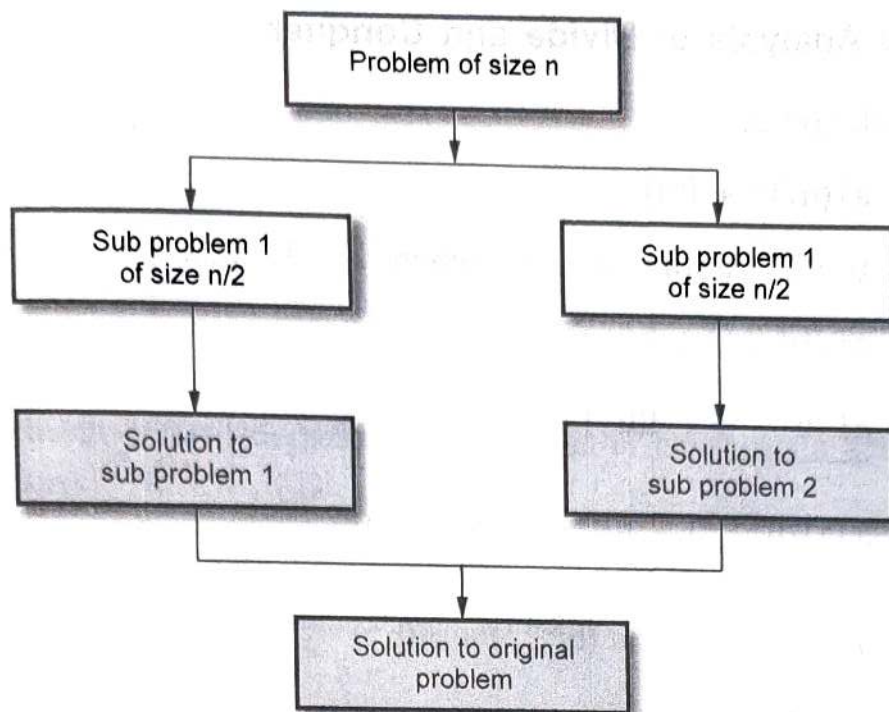


Fig. 3.1.1 Divide and conquer technique

For example, if we want to compute sum of n numbers then by divide and conquer we can solve the problem as

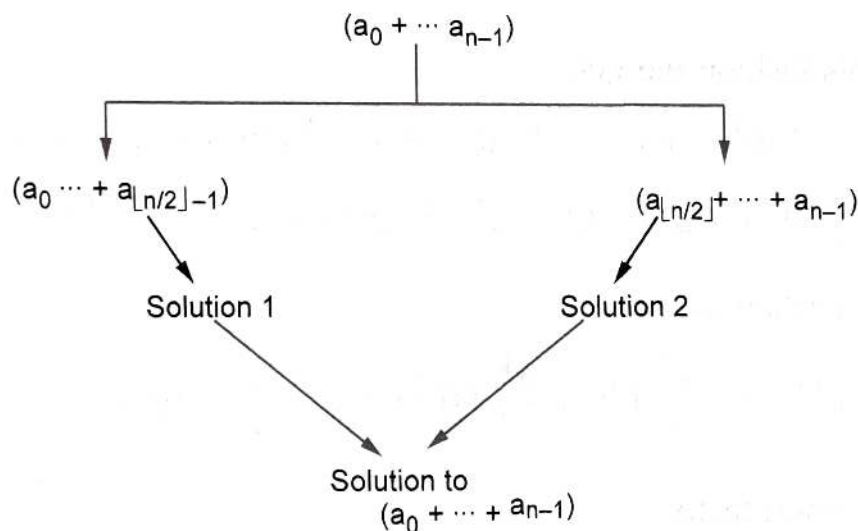
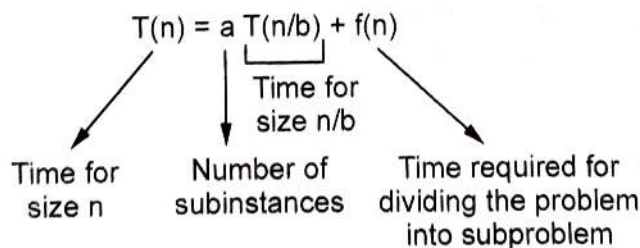


Fig. 3.1.2

If we want to divide a problem of size n into a size of n/b taking $f(n)$ time to divide and combine, then we can set up recurrence relation for obtaining time for size n is -



The above equation is called **general divide and conquer recurrence**. The order of growth of $T(n)$ depends upon the constants a , b and order of growth function $f(n)$.

Binary Search:-

Algorithm:-

1. $low \leftarrow 0$
2. $high \leftarrow n-1$
3. while ($low < high$) do
 - { $m \leftarrow (low + high) / 2$ // mid of array
 - if ($Key = A[m]$) then
 - return m
 - else if ($Key < A[m]$) then
 - $high \leftarrow m-1$ // search left sublist
 - else
 - $low \leftarrow m+1$ // search right sublist
 - }
 - return -1 // if element is not present in list

Analysis of Time Complexity:-

In the algorithm after one comparison the list of n elements is divided into $n/2$ sublists. The worst case efficiency is that the algorithm compares all the array elements for searching the desired element. In this method one comparison is made & based on this comparison array is divided each time in $n/2$ sub lists. Hence the worst case time complexity is given by

$$T(n) = T(n/2) + 1 \quad \text{for } n > 1$$

Time required to compare
left sublist or right sublist

one comparison made with
middle element

Finding complexity of Binary search using Master Method

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

Worst Case

On comparing with general equation i.e. $T(n) = aT\left(\frac{n}{b}\right) + f(n)$

$$a = 1, b = 2, f(n) = 1$$

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1$$

$$\therefore n^{\log_b a} = f(n) = 1$$

So, case 2 applies here

$$\text{i.e. } T(n) = O(n^{\log_b a} \cdot \log n)$$

$$T(n) = O(\log n)$$

However, it's common to write:-

$$T(n) = O(\log n)$$

This is because Big O notation is often used to give an upper bound on the time complexity.

Best Case

The best case for binary search occurs when the target element is found in the first iteration, i.e., when the target element is at the middle index of the array.

So, in this case, the algorithm only needs to perform one comparison to find the target element.

So, Time complexity of binary search in best case is.

$$O(1)$$

which means the algorithm takes constant time to find the target element.

Average Case of Binary Search

The average case for binary search occurs when the target element is found in the array, but not in the first iteration.

In this case, the algorithm performs $\log_2(n)$ comparisons on average, where n is the size of the array.

So, Time complexity of binary search in the average case is $O(\log n)$.

Binary search	Best	Average	Worst
Complexity	$O(1)$	$O(\log n)$	$O(\log n)$

Master Method

①

The problem is divided into $\{a\}$ number of subproblems each of size $\frac{n}{b}$ & need time $f(n)$ to combine the solution, then the running time $T(n)$ can be:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where $a \geq 1$ & $b > 1$

$f(n)$ is asymptotically positive function.

$\left(\frac{n}{b}\right)$ mean either $\lfloor \frac{n}{b} \rfloor$ or $\lceil \frac{n}{b} \rceil$

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then ~~then~~

$$T(n) = \Theta(n^{\log_b a})$$

2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$

3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$ & if $a f(n/b) \leq c f(n)$ for some constant $c < 1$ & all sufficiently large n , then $T(n) = \Theta(f(n))$

Q Solve the recurrence $T(n) = 9T\left(\frac{n}{3}\right) + n$

Sol. $a = 9$, $b = 3$, $f(n) = n$

$$\text{Now, } n^{\log_b a} = n^{\log_3 9} = n^2$$

$$f(n) = n^{2-1}$$

Hence, we apply case 1 & sol. is $\Theta(n^{\log_b a})$

i.e. $T(n) = \Theta(n^2)$

Q $T(n) = T\left(\frac{2n}{3}\right) + 1$

Sol. $a=1, b=\frac{3}{2}, f(n)=1$

Now, $n^{\log_a b} = n^{\log_{3/2} 1} = n^0 = 1$

$f(n) = n^{\log_a b}$

Hence case 2 applies

Thus, sol. is $T(n) = \Theta(n^{\log_a b} \log n)$
 $T(n) = \Theta(\log n)$

Q $T(n) = 16T\left(\frac{n}{4}\right) + n^3$

Sol. $a=16, b=4, f(n)=n^3$

Now, $n^{\log_a b} = n^{\log_{4} 16} = n^2$

$f(n) = n^{3-\epsilon} \quad (n^{\log_a b - \epsilon})$

So, case

Now check regularity condition i.e.

$\Rightarrow a f\left(\frac{n}{b}\right) \leq c f(n)$

$\Rightarrow 16 f\left(\frac{n^3}{4^3}\right) \leq c f(n^3)$

$\Rightarrow \frac{16 \cdot n^3}{64} \leq c n^3$

$\Rightarrow \frac{n^3}{4} \leq c n^3 \quad \text{for } c = \frac{1}{4} \quad \text{i.e. } c < 1$

So regularity condition holds

Hence by case 3, Sol. is $T(n) = \Theta(n^3)$

use 1:-

If $f(n) = O(n^{\log_a b - \epsilon})$ for some constant $\epsilon > 0$
Then $T(n) = O(n^{\log_a b}) \quad \{ n^{\log_a b} > f(n) \}$

Q Solve the recurrence $T(n) = 9T\left(\frac{n}{3}\right) + n$

Sol. Here $a = 9, b = 3, f(n) = n$

Now, first find $n^{\log_a b}$

$$n^{\log_3 9} = n^2$$

$$\text{So, } f(n) = O(n^{\log_3 9 - 1})$$

$$\text{i.e. } f(n) = O(n^{2-1})$$

So, case 1 applies & $T(n) = O(n^{\log_a b})$

$$\text{i.e. } \boxed{T(n) = O(n^{\log_3 9}) = O(n^2)}$$

$$\begin{cases} T(n) = aT\left(\frac{n}{b}\right) + f(n) \\ a = 9, b = 3, f(n) = n \end{cases}$$

$$\begin{cases} \text{compare } n^{\log_a b} \& f(n) \\ n^{\log_a b} > f(n) \end{cases}$$

Q Solve the recurrence $T(n) = 16T\left(\frac{n}{4}\right) + n$

Comparing above recurrence relation with general equation

$$\text{i.e. } T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

we get, $a = 16, b = 4, f(n) = n$

$$\text{Now find } n^{\log_a b} = n^{\log_4 16} = n^2$$

$$\text{Compare both } n^{\log_a b} \& f(n) \quad \{ n^{\log_4 16} > f(n) \}$$

So, case 1 applies here

$$\boxed{T(n) = O(n^{\log_4 16}) = O(n^2)}$$

Case 2:- If $f(n) = O(n^{\log_b a})$, then $T(n) = O(n^{\log_b a} \cdot \log n)$
 $\{ n^{\log_b a} = f(n) \}$

Q $T(n) = T\left(\frac{2n}{3}\right) + 1$

Sol. $T(n) = aT\left(\frac{n}{b}\right) + f(n)$

On comparing we get, $a = 1$, $b = \frac{3}{2}$, $f(n) = 1$

Now, $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$

$f(n) = n^{\log_b a}$

So case 2 applies here

Thus, $T(n) = O(n^{\log_b a} \cdot \log n)$

$T(n) = O(\log n)$

Q $T(n) = T\left(\frac{n}{3}\right) + 1$

Sol. Here, $a = 1$, $b = 3$, $f(n) = 1$

Now, $n^{\log_b a} = n^{\log_3 1} = n^0 = 1$

So, $f(n) = 1 = n^{\log_3 1}$

So case 2 applies here

So, solution is

$T(n) = O(n^{\log_b a} \cdot \log n)$

$T(n) = O(1 \cdot \log n)$

$T(n) = O(\log n)$

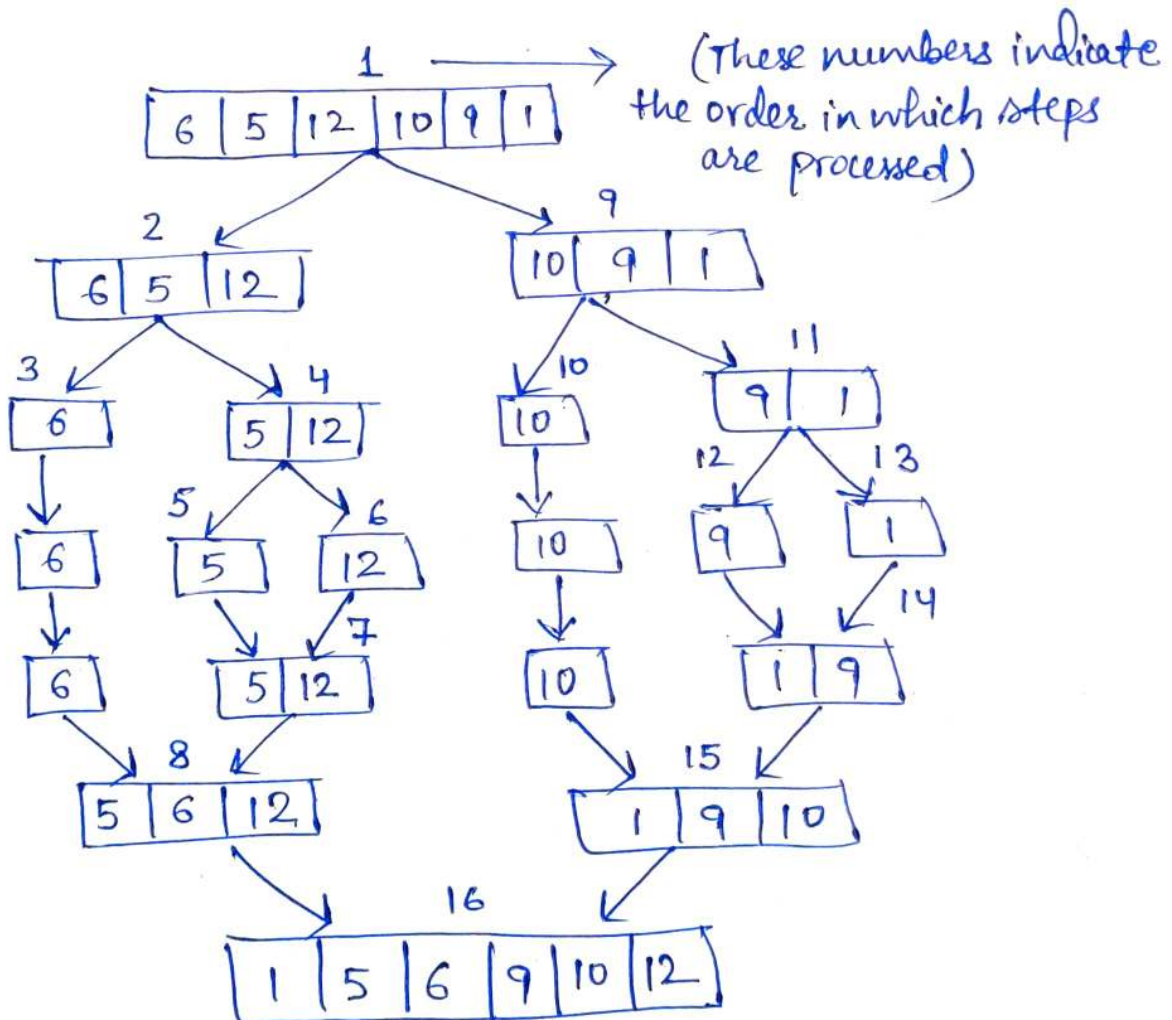
Merge Sort

Merge Sort is a divide & Conquer algorithm. It divides input array in two halves, call itself for the two halves & then merges the two sorted halves.

Functioning of Merge Sort:-

1. Find middle point to divide the array into approx. 2 halves.
2. Call mergesort for first half
3. Call mergesort for second half
4. Merge the two halves sorted in step 2 & 3.

for ex -



Merge (A, p, q, r)

1. $n_1 \leftarrow q - p + 1$
2. $n_2 \leftarrow r - q$
3. create arrays $L[1, 2, \dots, n_1+1]$ & $R[1, 2, \dots, n_2+1]$
4. for $i \leftarrow 1$ to n_1
5. do $L[i] \leftarrow A[p+i-1]$
6. for $j \leftarrow 1$ to n_2
7. do $R[j] \leftarrow A[q+j]$
8. $L[n_1+1] \leftarrow \infty$
9. $R[n_2+1] \leftarrow \infty$
10. $i \leftarrow 1$
11. $j \leftarrow 1$
12. for $k \leftarrow p$ to r
13. do if $L[i] \leq R[j]$
14. then $A[k] \leftarrow L[i]$
15. $i \leftarrow i+1$
16. else $A[k] \leftarrow R[j]$
17. $j \leftarrow j+1$

MergeSort (A, p, r)

1. if $p < r$
2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$
3. MergeSort (A, p, q)
4. MergeSort (A, q+1, r)
5. Merge (A, p, q, r)

1	2	3	4	5	6	7	8
4	5	7	2	1	3	6	2

$p=1, r=8$

Sol.

MergeSort(A, p, r) (A, 1, 8)

1. if $p < r$ ($1 < 8$) ✓
2. then $q \leftarrow \frac{p+r}{2} = q = 4$

3. MergeSort(A, p, q) (A, 1, 4)

4	5	7	2
---	---	---	---

1. if $p < r$ ($1 < 4$) ✓
2. then $q \leftarrow \frac{p+r}{2} = q = 2$

3. MergeSort(A, p, q) (A, 1, 2)

4	5
---	---

1. if $p < r$ ($1 < 2$) ✓
2. then $q \leftarrow \frac{p+r}{2} = q = 1$

3. MergeSort(A, p, q) (A, 1, 1)

4

1. if $p < r$ (~~$1 < 1$~~) ($1 < 1$) ✗

4. MergeSort(A, q+1, r) (A, 2, 2)

5

1. if $p < r$ ($2 < 2$) ✗

5. Merge(A, p, q, r) (A, 1, 1, 2)

① $n_1 \leftarrow q - p + 1 \Rightarrow n_1 = 1$

② $n_2 \leftarrow r - q \Rightarrow n_2 = 1$

L			R		
---	--	--	---	--	--

③ create arrays $L[1, 2, \dots, n_1+1]$ & $R[1, 2, \dots, n_2+1]$

④ for $i \leftarrow 1$ to n_1

⑤ (for $i=1$) $L[i] \leftarrow A[p+i-1] \Rightarrow L[1] \leftarrow A[1]$ L

4	
---	--

⑥ for $j \leftarrow 1$ to n_2

⑦ (for $j=1$) $R[j] \leftarrow A[q+j] \Rightarrow R[1] \leftarrow A[2]$ R

5	
---	--

⑧ $L[n_1+1] \leftarrow \infty$ $L[2] = \infty$

L	4	∞
---	---	----------

$$R[n_2+1] \leftarrow \infty \Rightarrow R[2] = \infty$$

5 | ∞

11

10 $i \leftarrow 1$

11 $j \leftarrow 1$

12 for $K \leftarrow p$ to n

13 do if $L[i] \leq R[j]$ ($4 < 5$) ✓

14 $A[K] \leftarrow L[i]$

4 | $\rightarrow$ A[K] array

15 $i \leftarrow i+1$ so, ($i=2$)

16 do if $L[2] \leq R[1]$ i.e. $L[2] = \infty$ $\infty \leq 5$ ✗
 $R[1] = 5$

16 ~~do~~ $A[K] \leftarrow R[j]$ 4 | 5

17 $j \leftarrow j+1 \Rightarrow j=2$

Analysis of Merge Sort :-

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T(n) = n \log_2 n + n$$

$$T(n) \approx n(\log_2 n + 1)$$

$$\boxed{T(n) = O(n \log_2 n)}$$

$$\left\{ \begin{array}{l} a=2, b=2, f(n)=n \\ n \log_b^a = n \log_2^2 = n \\ f(n) = n \log_b^1 \\ \therefore T(n) = O(n \log n) \end{array} \right.$$

In all cases complexity is same as all the steps of algo will be executed.

merge sort	Best	Average	worst
Complexity	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Quick Sort

(12)

Quick sort is based on divide & conquer approach.

Quicksort(A, p, r)

1. if $p < r$
2. then $q \leftarrow \text{partition}(A, p, r)$
3. Quicksort($A, p, q-1$)
4. Quicksort($A, q+1, r$)

Partition(A, p, r)

1. $x \leftarrow A[r]$
2. $i \leftarrow p-1$
3. for $j \leftarrow p$ to $r-1$
4. { if $A[j] \leq x$
5. $i \leftarrow i+1$
6. $A[i] \leftrightarrow A[j]$ }
7. $i = i+1$
8. $A[i] \leftrightarrow A[r]$
9. return i

Q

1	2	3	4	5	6	7	8
2	8	7	1	3	5	6	4

$p=1$ $r=8$

Sol.

$$i = p-1 = 1-1 = 0$$

$$j = p \text{ to } r-1 \text{ i.e. } 1 \text{ to } 7$$

Step 1. $j=1$ $x \leftarrow A[r] \therefore x \leftarrow A[8] = 4$

$$A[j] \leq x \quad A[1] \leq 4 \text{ i.e. } 2 \leq 4 \text{ true}$$

$$i = 0+1 = 1$$

$$A[i] \leftrightarrow A[j] \text{ i.e. } A[1] \text{ remains } 2$$

Step 2 $j=2, i=1$

$$A[2] \leq 4 \text{ i.e. } 8 \leq 4 \text{ False}$$

Step 3 $j=3, i=1$
 $A[3] \leq 4$ i.e. $7 \leq 4$ False

Step 4 $j=4, i=1$
 $A[4] \leq 4$ i.e. $1 \leq 4$ True

$i \leftarrow i+1, i=2$

Swap $A[2] \leftrightarrow A[4]$

i.e. $A[2]=1, A[4]=8$

1	2	3	4	5	6	7	8
2	1	7	8	3	5	6	4

Step 5:- $j=5, i=2$

$A[5] \leq 4$ i.e. $3 \leq 4$ True

$i=3$

$A[3] \leftrightarrow A[5]$ i.e. $A[3]=3$ & $A[5]=7$

1	2	3	4	5	6	7	8
2	1	3	8	7	5	6	4

Step 6 $j=6, i=3$

$A[6] \leq 4$ False

Step 7 $j=7, i=3$

$A[7] \leq 4$ False

Step 8 ~~$i=8$~~ , Now for loop $j \leftarrow 1$ to 7 terminate
 after this increment i
 $i=4$

then $A[i] \leftrightarrow A[j]$

$A[4] \leftrightarrow A[8]$

1	2	3	4	5	6	7	8
2	1	3	4	7	5	6	8

$q=4$ i.e. index 4

~~array~~

QuickSort (A, p, q-1) $\Rightarrow$ (A, 1, 3)

QuickSort (A, q+1, r) $\Rightarrow$ (A, 5, 8)

(13)

QuickSort (A, 1, 3)

p = 1, r = 3, x = 3

1	2	3
2	1	3

i = 0, j $\leftarrow$ 1 to 2 (p to r-1)

Step 1

j = 1

A[j] $\leq$ 3 i.e. A[1] $\leq$ 3

2 $\leq$ 3 True.

i = 0 + 1 = 1

A[1] $\leftrightarrow$ A[j] i.e. 2 remains same at its position

Step 2

j = 2, i = 1

A[2] $\leq$ 3 i.e. ~~2~~ 1 $\leq$ 3 True

i = 2

A[2] $\leftrightarrow$ A[2] i.e. 1 remains same at its position

Step 3

for loop terminate

i = 2 + 1 = 3

Swap A[i] $\leftrightarrow$ A[r]

A[3] $\leftrightarrow$ A[3]

q = 3

QuickSort (A, p, q-1) $\Rightarrow$ (A, 1, 2)

i = 0, p = 1, j = 1, x = 1

A[i] $\leq$ 1 2 $\leq$ 1 false.

i = 1 Swap A[i] $\leftrightarrow$ A[r]

A[1] $\leftrightarrow$ A[2]

1	2
---	---

1	2
2	1
p.	r

Quicksort (A, 5, 8)

$i = p-1, j = 5 \text{ to } 7$

$i = 4$

5	6	7	8
7	5	6	8
p			r

Comp

Step 1 $i = 4, j = 5, x = 8$

$A[5] \leq 8$ i.e. $7 \leq 8$ True

$i = 5$

$A[5] \leftrightarrow A[5]$

Step 2 $i = 5, j = 6$

$A[6] \leq 8$ i.e. $5 \leq 8$ ✓

$i = 6$

$A[6] \leftrightarrow A[6]$

Step 3 $i = 6, j = 7$

$A[7] \leq 8$ i.e. $6 \leq 8$ True

$i = 7$

$A[7] \leftrightarrow A[7]$

Step 4

$i = 8$

$A[8] \leftrightarrow A[8]$

$q = 8$

Quicksort (A, 5, 7)

Step 1 $i = p-1, j = 5 \text{ to } 6, x = 6$

5	6	7
7	5	6
p		r

$i = 4, j = 5$ $i = 4$

$A[5] \leq 6$ i.e. $7 \leq 6$ False

Step 2 $j = 6, i = 4$

$A[6] \leq 6$ i.e. $5 \leq 6$ True

$i = 4 + 1 = 5$

$A[5] \leftrightarrow A[6]$

5	6	7
5	7	6

Step 3

$i = 6$

$A[6] \leftrightarrow A[7]$

5	6	7
5	6	7

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Complexity Analysis of Quick Sort:-

1. Best Case:- (Split in the middle)

The recurrence relation for quick sort for obtaining best case time complexity is,

$$T(n) = \underbrace{T(n/2)} + \underbrace{T(n/2)} + n \rightarrow$$

Time required
to sort left
sub array

Time required
to sort right
sub array

Time required
for partitioning the
sub array

we get,

$$T(n) = 2T(n/2) + n$$

(Using master method)

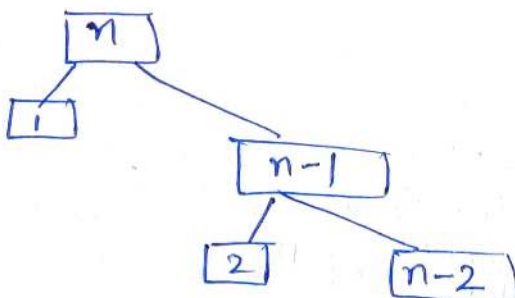
$$a=2, b=2, f(n)=n$$

$$n^{\log_a b} = n^{\log_2 2} = n$$

So, case 2 applies as $n^{\log_a b} = f(n)$

$$\therefore \boxed{T(n) = O(n \log n)}$$

2. Worst Case:- (when pivot is a max. or min. of all elements)



$$T(n) = T(n-1) + n$$

Solving the recurrence relation $T(n) = T(n-1) + n$ using substitution method, we get: -

$$T(n) = O(n^2)$$

3. Average Case: -

The recursion tree of Quick sort is ~~not~~ roughly balanced with an expected height of $O(\log n)$.

work done at each level of recursion tree is $O(n)$.

So, Total work done or Time complexity in average case:-

$$T(n) = O(n \cdot \log n)$$

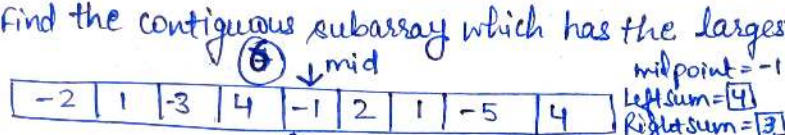
Quick sort	Best	Average	worst
Complexity	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Maximum Sum Subarray: —

Steps to find maximum Sum Subarray: —

- 1) Divide the given array in two halves.
- 2) Return the maximum of following three
 - a) Recursively calculate the maximum subarray sum in left half.
 - b) Recursively calculate the maximum subarray sum in right half.
 - c) Recursively calculate the maximum subarray crosses the midpoint.
 - i) Find the maximum sum starting from mid point and ending at some point on left of mid.
 - ii) Find the maximum sum starting from mid+1 & ending with some point on right of mid+1.
 - iii) Finally, combine the two & return.

Q Given an integer Array. Find the contiguous subarray which has the largest sum & return its sum.
 A=



$\therefore \text{max Sum} = \max(\text{Left Sum}, \text{Right Sum}, \text{CSS}) = 4$

mid element = -3
 Left Sum = 1, Right Sum = 4

Cross sum calculation:-

Left Sub Sum = $\max(-3, (-3+1), (-3+1+2))$
 LSS = -2
 RSS = $\max(4, 4+(-1)) = 4$
 CSS = $-2 + 4 = 2$

mid element = 1

Left Sum = 1

Right Sum = -3

Cross sum calculation:-

Left Sub Sum = $\max(1, 1+(-2)) = 1$

Right Sub Sum = -3

Cross Sum = LSS + RSS

CSS = $1 + (-3) = -2$

$\therefore \text{max Sum} = \max(\text{LSS}, \text{RSS}, \text{CSS})$

= 1



mid element = -2

Left Sum = -2

Right Sum = 1

Cross sum = LSS + RSS

CSS = $-2 + 1 = -1$

$\text{max Sum} = \max(\text{LSS}, \text{RSS}, \text{CSS}) = 1$

(-2, 1, -1)

Cross sum calculation:-

LSS = $\max(-1, -1+4, -1+4+3, -1+4+3+1)$

RSS = $\max(2, 2+1, 2+1+(-5), 2+1+(-5)+4)$

CSS = LSS + RSS = 3 + 3 = 6

max = $\max(4, 4, 6)$

index of mid = $\frac{\text{big} + \text{end}}{2}$

left half = beg to mid

Right half = mid + 1 to end

mid element = 1

Left Sum = 3, Right Sum = 4

Cross sum calculation:-

LSS = $\max(1, 1+2) = 3$

RSS = $\max(-5, -5+4) = -1$

CSS = LSS + RSS = 3 + (-1) = 2

$\therefore \text{max} = 3$

Cross sum = LSS + RSS = -1

max sum = $\max(-5, 4, -1)$

max = 4

General formula:-

1. First of all Find mid element
2. Find left Sum coming from left array
3. Find right Sum coming from right array
4. Then calculate Cross sum. For this first find Left Sub Sum
 $\text{Left Sub Sum (LSS)} = \max(\text{mid element}, \text{mid}-1, \text{mid}-2, \dots)$
 $\text{Right Sub Sum (RSS)} = \max(\text{mid}+1 \text{ element}, \text{mid}+2, \dots)$
 Then find Cross Sub Sum (CSS) = LSS + RSS
5. Finally find maximum sum = $\max(\text{LSS}, \text{RSS}, \text{CSS})$

3.6 Strassen's Matrix Multiplication

Suppose we want to multiply two matrices A and B each of size N i.e.

$$C = A \times B \quad \text{then}$$

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \times \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

The multiplication gives

$$C_{11} = A_{11} \times B_{11} + A_{12} \times B_{21}$$

$$C_{12} = A_{11} \times B_{12} + A_{12} \times B_{22}$$

$$C_{21} = A_{21} \times B_{11} + A_{22} \times B_{21}$$

$$C_{22} = A_{21} \times B_{12} + A_{22} \times B_{22}$$

Thus to accomplish 2×2 matrix multiplication there are total 8 multiplications and 4 additions.

To accomplish this multiplication we can write the following algorithm for the same.

Algorithm Mat_Mul (A,B,C,n)

```
{
    for i:=1 to n do
        for j:=1 to n do
            C[i,j] := 0;
            for k:=1 to n do
                C[i,j] := C[i,j] + A[i,k] × B[k,j] ;
            }
        }
}
```

The time complexity of above algorithm turns to be

$$O(n \times n \times n) = O(n^3)$$

Strassen showed that 2×2 matrix multiplication can be accomplished in 7 multiplication and 18 additions or subtractions.

The divide and conquer approach can be used for implementing Strassen's matrix multiplication.

- **Divide** : Divide matrices into sub-matrices : A_0, A_1, A_2 etc.
- **Conquer** : Use a group of matrix multiply equations.
- **Combine**
Recursively multiply sub-matrices and get the final result of multiplication after performing required additions or subtractions.

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \times \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$$

$$S_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$S_2 = (A_{21} + A_{22}) \times B_{11}$$

$$S_3 = A_{11} \times (B_{12} - B_{22})$$

$$S_4 = A_{22} \times (B_{21} - B_{11})$$

$$S_5 = (A_{11} + A_{12}) \times B_{22}$$

$$S_6 = (A_{21} - A_{11}) \times (B_{11} + B_{12})$$

$$S_7 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$C_{11} = S_1 + S_4 - S_5 + S_7$$

$$C_{12} = S_3 + S_5$$

$$C_{21} = S_2 + S_4$$

$$C_{22} = S_1 + S_3 - S_2 + S_6$$

Now we will compare the actual our traditional matrix multiplication procedure with Strassen's procedure. In Strassen's multiplication

$$C_{11} = S_1 + S_4 - S_5 + S_7$$

$$= (A_{11} + A_{22})(B_{11} + B_{22}) + A_{22} \times (B_{21} - B_{11}) - (A_{11} + A_{12}) \times B_{22} + (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$= A_{11} B_{11} + A_{11} B_{22} + A_{22} B_{11} + A_{22} B_{22} + A_{22} B_{21} - A_{22} B_{11} - A_{11} B_{22} - A_{12} B_{22} + A_{12} B_{21} + A_{12} B_{22} - A_{22} B_{21} - A_{22} B_{22}$$

$$= A_{11} B_{11} + A_{12} B_{21}$$

3.6.1 Algorithm

Here we are dividing matrices in sub-matrices and recursively multiplying sub-matrices

Algorithm St_Mul(int *A, int *B, int *C, int n)

```

{
  if (n == 1) then
  {
    (*C) = (*C) + (*A) * (*B);
  }
  else
  {
    St_Mul(A, B, C, n/4);
    St_Mul(A, B + (n/4), C + (n/4), n/4);
    St_Mul(A + 2 * (n/4), B, C + 2 * (n/4), n/4);
    St_Mul(A + 2 * (n/4), B + (n/4), C + 3 * (n/4), n/4);
    St_Mul(A + (n/4), B + 2 * (n/4), C, n/4);
    St_Mul(A + (n/4), B + 3 * (n/4), C + (n/4), n/4);
    St_Mul(A + 3 * (n/4), B + 2 * (n/4), C + 2 * (n/4), n/4);
    St_Mul(A + 3 * (n/4), B + 3 * (n/4), C + 3 * (n/4), n/4);
  }
}

```

3.6.2 Analysis of Algorithm

$$T(1) = 1$$

assume $n = 2^k$

$$T(n) = 7 T(n/2)$$

$$T(n) = 7^k T(n/2^k)$$

$$T(n) = 7^{\log n}$$

$$T(n) = n^{\log 7} = n^{2.81}$$

Thus divide and conquer is an algorithmic strategy having with the principle idea of dividing the problem into subproblems. Then solution to these subproblems is obtained in order to get the final solution for the given problem.

Example 3.6.2 Apply Strassen's method to multiply the following two matrices

$$\begin{bmatrix} 2 & 3 & 4 & 5 \\ 6 & 3 & 8 & 2 \\ 3 & 8 & 0 & 6 \\ 5 & 4 & 3 & 2 \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} 6 & 7 & 8 & 9 \\ 2 & 0 & 4 & 5 \\ 7 & 3 & 8 & 0 \\ 9 & 2 & 7 & 6 \end{bmatrix}$$

GBTU : Nov.-10

Solution : The given matrix is of order 4×4 . Hence we will subdivide it in 2×2 submatrices. Hence we will compute $S_1, S_2, S_3, S_4, S_5, S_6, S_7$.

Let,

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{bmatrix} \begin{bmatrix} 2 & 3 \\ 6 & 3 \end{bmatrix} & \begin{bmatrix} 4 & 5 \\ 8 & 2 \end{bmatrix} \\ \begin{bmatrix} 3 & 8 \\ 5 & 4 \end{bmatrix} & \begin{bmatrix} 0 & 6 \\ 3 & 2 \end{bmatrix} \end{bmatrix} \times \begin{bmatrix} \begin{bmatrix} 6 & 7 \\ 2 & 0 \end{bmatrix} & \begin{bmatrix} 8 & 9 \\ 4 & 5 \end{bmatrix} \\ \begin{bmatrix} 7 & 3 \\ 9 & 2 \end{bmatrix} & \begin{bmatrix} 8 & 0 \\ 7 & 6 \end{bmatrix} \end{bmatrix}$$

Now

$$\begin{aligned} S_1 &= (A_{11} + A_{22}) (B_{11} + B_{22}) \\ &= \left[\begin{bmatrix} 2 & 3 \\ 6 & 3 \end{bmatrix} + \begin{bmatrix} 0 & 6 \\ 3 & 2 \end{bmatrix} \right] \times \left[\begin{bmatrix} 6 & 7 \\ 2 & 0 \end{bmatrix} + \begin{bmatrix} 8 & 9 \\ 4 & 5 \end{bmatrix} \right] \\ &= \begin{bmatrix} 2 & 9 \\ 9 & 5 \end{bmatrix} \times \begin{bmatrix} 14 & 7 \\ 9 & 6 \end{bmatrix} \\ &= \begin{bmatrix} 28+81 & 14+54 \\ 126+45 & 63+30 \end{bmatrix} \end{aligned}$$

$$S_1 = \begin{bmatrix} 109 & 68 \\ 171 & 93 \end{bmatrix}$$

$$\begin{aligned} S_2 &= (A_{21} + A_{22}) \times B_{11} \\ &= \left[\begin{bmatrix} 3 & 8 \\ 5 & 4 \end{bmatrix} + \begin{bmatrix} 0 & 6 \\ 3 & 2 \end{bmatrix} \right] \times \begin{bmatrix} 6 & 7 \\ 2 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 3 & 14 \\ 8 & 6 \end{bmatrix} \times \begin{bmatrix} 6 & 7 \\ 2 & 0 \end{bmatrix} \\ &= \begin{bmatrix} 9-28 & 21+0 \\ 48+12 & 56+0 \end{bmatrix} \end{aligned}$$

$$S_2 = \begin{bmatrix} 37 & 21 \\ 60 & 56 \end{bmatrix}$$

$$\begin{aligned} S_3 &= A_{11} \times (B_{12} - B_{22}) \\ &= \begin{bmatrix} 2 & 3 \\ 6 & 3 \end{bmatrix} \times \left[\begin{bmatrix} 8 & 9 \\ 4 & 5 \end{bmatrix} - \begin{bmatrix} 8 & 0 \\ 7 & 6 \end{bmatrix} \right] \end{aligned}$$

$$S_3 = \begin{bmatrix} 2 & 3 \\ 6 & 3 \end{bmatrix} \times \begin{bmatrix} 0 & 9 \\ -3 & -1 \end{bmatrix}$$

$$S_3 = \begin{bmatrix} 0+9 & 18-3 \\ 0-9 & 54-3 \end{bmatrix}$$

$$S_3 = \begin{bmatrix} -9 & 15 \\ -9 & 51 \end{bmatrix}$$

$$\begin{aligned} S_4 &= A_{22} \times (B_{21} - B_{11}) \\ &= \begin{bmatrix} 0 & 6 \\ 3 & 2 \end{bmatrix} \times \left[\begin{bmatrix} 7 & 3 \\ 9 & 2 \end{bmatrix} - \begin{bmatrix} 6 & 7 \\ 2 & 0 \end{bmatrix} \right] \end{aligned}$$

$$= \begin{bmatrix} 0 & 6 \\ 3 & 2 \end{bmatrix} \times \begin{bmatrix} 1 & -4 \\ 7 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 0 + 42 & 0 + 12 \\ 3 + 14 & -12 + 4 \end{bmatrix}$$

$$S_4 = \begin{bmatrix} 42 & 12 \\ 17 & -8 \end{bmatrix}$$

$$\begin{aligned} S_5 &= (A_{11} + A_{12}) \times B_{22} \\ &= \left[\begin{bmatrix} 2 & 3 \\ 6 & 3 \end{bmatrix} + \begin{bmatrix} 4 & 5 \\ 8 & 2 \end{bmatrix} \right] \times \begin{bmatrix} 8 & 0 \\ 7 & 6 \end{bmatrix} \end{aligned}$$

$$S_5 = \begin{bmatrix} 6 & 8 \\ 14 & 5 \end{bmatrix} \times \begin{bmatrix} 8 & 0 \\ 7 & 6 \end{bmatrix} = \begin{bmatrix} 48+56 & 0+48 \\ 112+35 & 0+30 \end{bmatrix}$$

$$S_5 = \begin{bmatrix} 104 & 48 \\ 147 & 30 \end{bmatrix}$$

$$\begin{aligned} S_6 &= (A_{21} - A_{11}) \times (B_{11} + B_{12}) \\ &= \left[\begin{bmatrix} 3 & 8 \\ 5 & 4 \end{bmatrix} - \begin{bmatrix} 2 & 3 \\ 6 & 3 \end{bmatrix} \right] \times \left[\begin{bmatrix} 6 & 7 \\ 8 & 9 \end{bmatrix} + \begin{bmatrix} 8 & 9 \\ 7 & 6 \end{bmatrix} \right] \end{aligned}$$

$$= \begin{bmatrix} 1 & 5 \\ -1 & 1 \end{bmatrix} \times \begin{bmatrix} 14 & 16 \\ 6 & 5 \end{bmatrix}$$

$$= \begin{bmatrix} 14+30 & 16+25 \\ -14+6 & -16+5 \end{bmatrix}$$

$$S_6 = \begin{bmatrix} 44 & 41 \\ -8 & -11 \end{bmatrix}$$

$$S_7 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$= \left[\begin{bmatrix} 4 & 5 \\ 8 & 2 \end{bmatrix} - \begin{bmatrix} 0 & 6 \\ 3 & 2 \end{bmatrix} \right] \times \left[\begin{bmatrix} 7 & 3 \\ 9 & 2 \end{bmatrix} + \begin{bmatrix} 8 & 0 \\ 7 & 6 \end{bmatrix} \right]$$

$$= \begin{bmatrix} 4 & -1 \\ 5 & 0 \end{bmatrix} \times \begin{bmatrix} 15 & 3 \\ 16 & 8 \end{bmatrix}$$

$$= \begin{bmatrix} 60+(-16) & 12-8 \\ 75+0 & 15+0 \end{bmatrix}$$

$$S_7 = \begin{bmatrix} 44 & 4 \\ 75 & 15 \end{bmatrix}$$

$$C_{11} = S_1 + S_4 - S_5 + S_7$$

$$= \begin{bmatrix} 109 & 68 \\ 171 & 93 \end{bmatrix} + \begin{bmatrix} 42 & 12 \\ 17 & -8 \end{bmatrix} - \begin{bmatrix} 104 & 48 \\ 147 & 30 \end{bmatrix} + \begin{bmatrix} 44 & 4 \\ 75 & 15 \end{bmatrix}$$

$$C_{11} = \begin{bmatrix} 91 & 36 \\ 116 & 70 \end{bmatrix}$$

$$C_{12} = S_3 + S_5$$

$$= \begin{bmatrix} -9 & 15 \\ -9 & 51 \end{bmatrix} + \begin{bmatrix} 104 & 48 \\ 147 & 30 \end{bmatrix}$$

$$C_{12} = \begin{bmatrix} 95 & 63 \\ 138 & 81 \end{bmatrix}$$

$$C_{21} = S_2 + S_4$$

$$= \begin{bmatrix} 37 & 21 \\ 60 & 56 \end{bmatrix} + \begin{bmatrix} 42 & 12 \\ 17 & -8 \end{bmatrix}$$

$$C_{21} = \begin{bmatrix} 79 & 33 \\ 77 & 48 \end{bmatrix}$$

$$\begin{aligned}
 C_{22} &= S_1 + S_3 - S_2 + S_6 \\
 &= \begin{bmatrix} 109 & 68 \\ 171 & 93 \end{bmatrix} + \begin{bmatrix} -9 & 15 \\ -9 & 51 \end{bmatrix} - \begin{bmatrix} 37 & 21 \\ 60 & 56 \end{bmatrix} + \begin{bmatrix} 44 & 41 \\ -8 & -11 \end{bmatrix} \\
 C_{22} &= \begin{bmatrix} 107 & 103 \\ 94 & 77 \end{bmatrix}
 \end{aligned}$$

The final product matrix C will be -

$$\begin{aligned}
 &= \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} \\
 &= \begin{bmatrix} \begin{bmatrix} 91 & 36 \\ 116 & 70 \\ 79 & 33 \\ 77 & 48 \end{bmatrix} & \begin{bmatrix} 95 & 63 \\ 138 & 81 \\ 107 & 103 \\ 94 & 77 \end{bmatrix} \end{bmatrix} \\
 C &= \begin{bmatrix} 91 & 36 & 95 & 63 \\ 116 & 70 & 138 & 81 \\ 79 & 33 & 107 & 103 \\ 77 & 48 & 94 & 77 \end{bmatrix}
 \end{aligned}$$

3.7 Finding Minimum and Maximum

First of all we will discuss an algorithm of finding the minimum element from the array of n numbers.

```

Algorithm Minimum_val (A[1.....n])
{
    // Problem Description : This algorithm is to
    // find minimum value from array A of n
    // elements
    min ← A[1] // Assuming first element as min.
    for (i ← 2 to n) do
    {
        if (min > A[i]) then
            min ← A[i] //set new min value
    }
    return min
}

```

Thus we require total (n-1) comparison to obtain minimum value. Similarly we can obtain the maximum value from an array A in (n-1) comparison. Here is an algorithm for finding maximum element.

Algorithm Maximum_val (A[1.....n])

```
{
    // Problem Description : This algorithm is to find maximum value from
    // array A of n elements
    max ← A[1]
    for (i ← 2 to n) do
    {
        if [A[i] > max) then
            max ← A[i]
        }
    }
    return max
}
```

We can obtain maximum and minimum values from an array simultaneously using following algorithm.

Algorithm Max_Min (A[1.....n], Max, Min)

```
{
    // Problem Description : This algorithm obtains min and max values simultaneously.
    Max ← Min ← A[1]
    for (i ← 2 to n) do
    {
        if (A[i] > Max) then
            Max ← A[i] // obtaining maximum value
        if (A[i] < Min) then
            Min ← A[i] // obtaining minimum value.
        }
    }
}
```

Thus we get maximum element in **max** and minimum element in **min** variable. Let us derive this algorithm for some example. Consider an array

1	2	3	4	5	6	7	8	9
50	40	- 5	- 9	45	90	65	25	75

Step 1 :

1	2	3	4	5	6	7	8	9
50	40	- 5	- 9	45	90	65	25	75

↑

Min

Max

Step 2 :

Now from index 2 to n we will compare an array element with Min and Max value.

1	2	3	4	5	6	7	8	9	Min = 40
50	40	- 5	- 9	45	90	65	25	75	Max = 50

Step 3 :

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Min = -5

Max = 50

Step 4 :

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Min = -9

Max = 50

Step 5 :

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Min = -9

Max = 45

Step 6 :

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Min = -9

Max = 90

Step 7 :

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Min = -9

Max = 90

Step 8 :

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Min = -9

Max = 90

Step 9 :

1	2	3	4	5	6	7	8	9
50	40	-5	-9	45	90	65	25	75

Min = -9

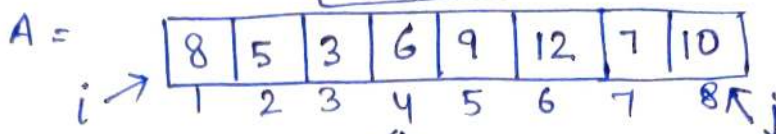
Max = 90

Step 10 :

As we have reached at n^{th} location of the array we will terminate the procedure of finding minimum and maximum values and print minimum value as -9 and maximum value as 90.

finding minimum & maximum in an array using divide & conquer.

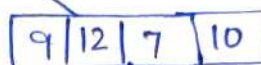
$\begin{bmatrix} \text{Max} = 12 \\ \text{Min} = 3 \end{bmatrix} \rightarrow \text{Final Solution}$



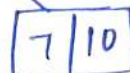
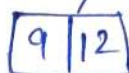
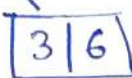
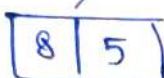
$$\text{mid} = \left\lfloor \frac{i+j}{2} \right\rfloor$$

$$\text{mid} = \left\lfloor \frac{1+8}{2} \right\rfloor = 4$$

Max = 8
Min = 3



Max = 12
Min = 7



Max 8
Min 5

Max 6
Min 3

Max 12
Min 9

Max 10
Min 7

Compare both max & min
for upper step
i.e. from max=8 & max=6
max for upper step is 8
& min is 3

Complexity Analysis:-

$$T(n) = 2T\left(\frac{n}{2}\right) + 2$$

For breaking
array into approx 2 halves

comparing 1 min & 1
max with other min
& max at each
step

Solving this using master's method:-

$$a=2, b=2, f(n)=2$$

$$n^{\log_a b} = n^{\log_2 2} = n$$

$$(i.e. n^{\log_a b} > f(n))$$

So case 1 applies here

$$\boxed{T(n) = O(n^{\log_a b}) = O(n)}$$

Convex-Hull :-

→ ढाँचा / outer covering

- A figure is convex if every line segments drawn between any two points inside the figure lies entirely inside the figure.
- A figure that is not convex is called a concave figure.



convex figures



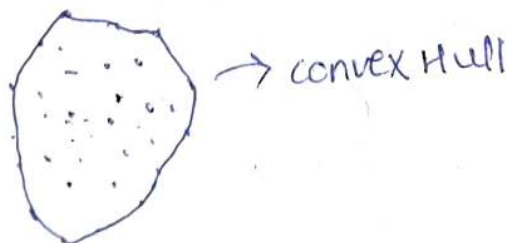
Concave figures

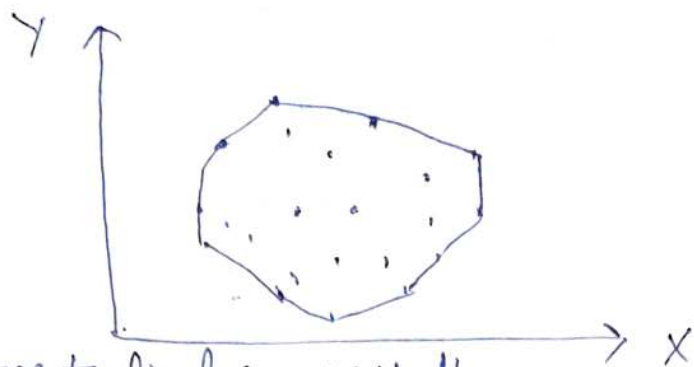
- In convex polygon any two points inside the polygon can be connected by a line segment that doesn't cross the polygon.

- Given a set $S = \{P_1, P_2, \dots, P_n\}$ of points in the plane, the convex hull of a set S of points is the smallest convex polygon P for which each point in S is either on the boundary of P or in its interior.

- We denote the convex hull of S by $CH(S)$.

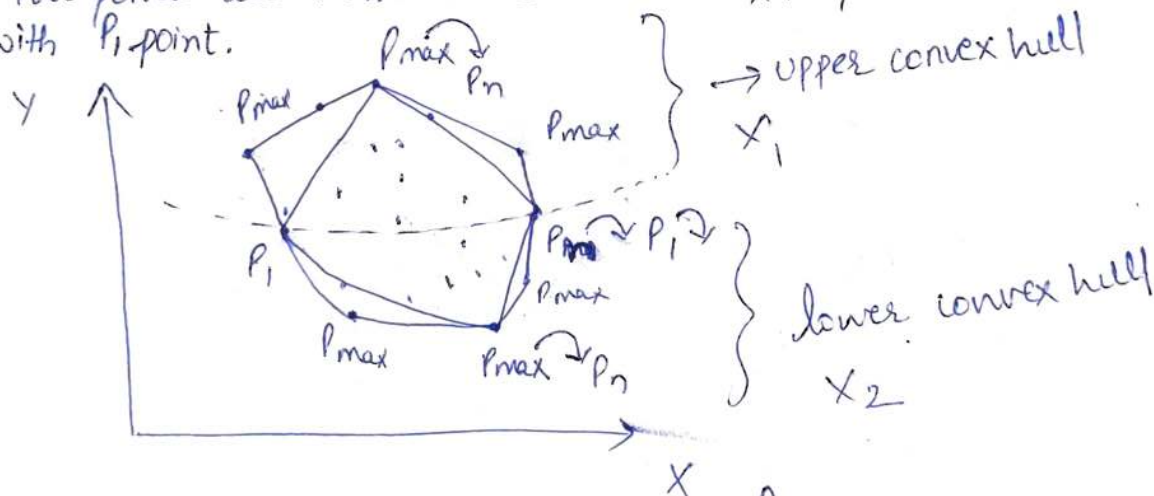
- Intersection of convex sets is convex, so the convex hull is a convex set.





Process to find convex Hull

1. Partition X into half X_1 & X_2 on x -coordinate.
2. If X_1 is empty, then upper hull is simply line with end points P_1 & P_n .
3. If X_1 is not empty the algorithm find P_{max} in X_1 which is farthest from line $P_1 P_n$.
4. Algo identifies all point of X_1 that are left of line $P_1 P_{max}$.
5. If two points are P_{max} then we consider P_{max} the point which makes larger angle with P_1 point.



After this remove inner lines

